

OpenRouter API Tutorial

In This notebook we will take a look at OpenRouter.ai, a platform that lets you interact with multiple AI models (like ChatGPT, Claude, etc.) through a single API.

Step 1: Get Your API Key

Go to OpenRouter.ai and sign up.

Click on "Keys" in the dashboard.

Copy your API key (it looks like sk-or-v1-...).

 Keep this key secret! Never share it publicly.

Step 2: Install Python & Required Libraries

You need Python (3.6+) and the requests library.

Install requests (if you don't have it):

```
In [2]: ! pip install requests
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (2.32.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests) (2025.6.15)
```

Step 3: Basic API Request

Let's make a simple request to OpenRouter.

```
In [22]: import requests

API_KEY = "your-api-key-here" # 🔑 Replace with your key!
API_URL = "https://openrouter.ai/api/v1/chat/completions"

headers = {
    "Authorization": f"Bearer {API_KEY}",
    "Content-Type": "application/json"
}

data = {
    "model": "deepseek/deepseek-chat:free", # 📝 Choose any model
    "messages": [
        {"role": "user", "content": "Tell me a joke about cats!"}
    ]
}

response = requests.post(API_URL, headers=headers, json=data)

if response.status_code == 200:
    answer = response.json()
    print("🤖 AI Response:", answer["choices"][0]["message"]["content"])
else:
    print("❌ Error:", response.text)
```

🤖 AI Response: Sure, here's a cat-themed joke for you:

Why don't cats play poker in the jungle?

Too many cheetahs! 🐾🐾🐾

What's happening?

We send a POST request to OpenRouter.

We specify the AI model (gpt-3.5-turbo).

We pass a user message (Tell me a joke about cats!).

The AI responds, and we print it.

Step 4: Customize Your Request

Change the AI Model

OpenRouter supports many models. Try these:

anthropic/claude-3-opus (Claude 3)

google/gemini-pro (Gemini)

mistralai/mistral-7b-instruct (Mistral)

Add a System Prompt (AI Personality)

```
In [4]: data = {
    "model": "deepseek/deepseek-chat:free",
    "messages": [
        {"role": "system", "content": "You are a funny comedian."}, #  AI personality
        {"role": "user", "content": "Tell me a joke about dogs!"}
    ]
}
```

Step 5: Error Handling

Sometimes the API fails. Handle errors like this:

```
In [ ]: response = requests.post(API_URL, headers=headers, json=data)

if response.status_code == 200:
    print("✅ Success:", response.json())
elif response.status_code == 401:
    print("🔒 Error: Invalid API Key!")
elif response.status_code == 429:
    print("🚫 Error: Too many requests! Slow down.")
else:
    print("❌ Unknown Error:", response.text)
```

Recap

```
In [5]: # Our secret password to talk to the robot
API_KEY = ''

# The robot's address on the internet
API_URL = 'https://openrouter.ai/api/v1/chat/completions'

# How we introduce ourselves to the robot
headers = {
    'Authorization': f'Bearer {API_KEY}', # Showing our secret password
    'Content-Type': 'application/json'    # Telling it we're sending text
}

your_question = 'what do you know about Shiraz?'

# Prepare what to send
message_to_send = {
    "model": "deepseek/deepseek-chat:free",
    "messages": [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": your_question}
    ]
}

# Send the question and get the answer
response = requests.post(API_URL, headers=headers, json=message_to_send)

# Show the answer
if response.status_code == 200:
    answer = response.json()
    print("🤖 Answer:", answer["choices"][0]["message"]["content"])
else:
    print("❌ Error:", response.status_code, response.text)
```

🤖 Answer: Shiraz is a city in southwestern Iran, known for its rich history, culture, and poetry. It is the capital of Fars Province and is situated in the Zagros Mountains. Shiraz is famous for its gardens, historical monuments, and as the birthplace of renowned Persian poets such as Hafez and Saadi. The city is also a significant cultural and economic center in Iran.

Next Steps:

- ◆ Try different models (GPT-4, Claude, etc.)
- ◆ Build a chatbot with memory
- ◆ Explore OpenRouter's docs for advanced features

Happy coding! 🚀

```
In [23]: models_url = "https://openrouter.ai/api/v1/models"
response = requests.get(models_url)

if response.ok:
    models = response.json()["data"]
    print("Available Models:")
    for model in models[:40]: # Show first 40
        # Inspect the model dictionary to find the correct key for the provider
        #print(model) # Uncomment this line to see the structure of the model dictionary
        print(f"- {model['id']}") # Use .get to avoid KeyError and display provider
else:
    print("Failed to fetch models.")
```

Available Models:

- inception/mercury)
- morph/morph-v2)
- mistralai/mistral-small-3.2-24b-instruct:free)
- mistralai/mistral-small-3.2-24b-instruct)
- minimax/minimax-m1:extended)
- minimax/minimax-m1)
- google/gemini-2.5-flash-lite-preview-06-17)
- google/gemini-2.5-flash)
- google/gemini-2.5-pro)
- moonshotai/kimi-dev-72b:free)
- openai/o3-pro)
- x-ai/grok-3-mini)
- x-ai/grok-3)
- mistralai/magistral-small-2506)
- mistralai/magistral-medium-2506)
- mistralai/magistral-medium-2506:thinking)
- google/gemini-2.5-pro-preview)
- sentientagi/dobby-mini-unhinged-plus-llama-3.1-8b)
- deepseek/deepseek-r1-distill-qwen-7b)
- deepseek/deepseek-r1-0528-qwen3-8b:free)
- deepseek/deepseek-r1-0528-qwen3-8b)
- deepseek/deepseek-r1-0528:free)
- deepseek/deepseek-r1-0528)
- sarvamai/sarvam-m:free)
- thedrummer/valkyrie-49b-v1)
- anthropic/claude-opus-4)
- anthropic/claude-sonnet-4)
- mistralai/devstral-small:free)
- mistralai/devstral-small)
- google/gemma-3n-e4b-it:free)
- google/gemma-3n-e4b-it)
- google/gemini-2.5-flash-preview-05-20)
- google/gemini-2.5-flash-preview-05-20:thinking)
- openai/codex-mini)
- mistralai/mistral-medium-3)
- google/gemini-2.5-pro-preview-05-06)
- arcee-ai/caller-large)
- arcee-ai/spotlight)
- arcee-ai/maestro-reasoning)
- arcee-ai/virtuoso-large)